COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Cloud Specialist [D68444-15-2025-23]

Unità Formativa : Programmazione Python

Docente: Massimo PAPA

Titolo argomento: Soluzione esercizi sulle classi (OOP) (3)

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Soluzione esercizi sulle classi (OOP)

Indicazioni sulla consegna **3**
Struttura del codice sorgente 4
Esercizi sulle classi (OOP) **6**
Primo Esercizio 6
Secondo Esercizio 6
Terzo Esercizio 6
Esercizi sull'ereditarietà **7**
Quarto Esercizio 7
Quinto Esercizio 7
Esercizi sul polimorfismo e sull'import **9**
Sesto Esercizio: Polimorfismo con oggetti veicolo 9
Settimo Esercizio: Polimorfismo con operatori e collezioni eterogenee 10
Ottavo Esercizio: Sistema modulare per gestione biblioteca 11
Soluzione esercizi sulle classi (OOP) **14**
Primo Esercizio 14
Secondo Esercizio 17
Terzo Esercizio 20
Soluzione esercizi sull'ereditarietà **23**
Quarto Esercizio 23
Quinto Esercizio 29
Soluzione esercizi sul polimorfismo e sull'import **33**
Sesto Esercizio 33
Settimo Esercizio 36
Ottavo Esercizio 39
FILE: media.py 39
FILE: biblioteca.py 40
FILE: main.py 41

Indicazioni sulla consegna

Implementare gli algoritmi risolutivi dei seguenti esercizi codificandoli in linguaggio python. Partire analizzando il problema seguendo la seguente traccia:

- Quali sono gli input del problema?
- Quali sono gli output?
- Suddividi il problema in problemi più semplici e se lo ritieni opportuno descrivi la soluzione di ogni sottoproblema con un flow-chart
- Andrai a rappresentare ogni algoritmo che risolve un sottoproblema come una funzione
- La funzione restituisce un valore? Se si di che tipo?
- Ogni funzione accetta una lista di parametri formali? Se si di che tipo?
- Esegui la codifica partendo dal programma principale, al suo interno vai a richiamare le funzioni che poi andrai successivamente a definire.
- Continua la codifica dichiarando e definendo tutte le funzioni prima del programma principale che richiama tutte le altre. In testa alle funzioni scrivi un commento che descrive la funzione stessa , cosa restituisce e quali sono i parametri formali.
- Scrivi la sezione dell'inizializzazione variabili scrivendo in un commento il significa della variabile stessa
- Verifica la codifica utilizzando input di test, cercando di provare anche i casi limite.

Struttura del codice sorgente

Relativamente alle indicazioni di scrittura del codice, utilizza il seguente schema generale:

```
'''
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Testo esercizio

'''

##
## Funzioni:
##


def fun(param1: int, param2: float) -> int:
  '''
     Funzione: fun
      Template per costruire le funzioni
      Parametri formali:
          int Param1 -> descrizione Param1
          float Param2 -> descrizione Param2
      Valore di ritorno:
          int -> descrizione valore di ritorno
  '''
  retValue = 5 # Valore di ritorno della funzione

  return retValue


'''
    Programma principale
    Descrizione sintetica funzionalità
    del programma principale.
'''

# Sezione di input Dati

# Inizializzazioni variabili

# Elaborazione
```

```
# Eventuali sotto processi di Elaborazione

# Sezione di output
```

Esercizi sulle classi (OOP)

Primo Esercizio

Creare una classe Insegnante con attributi nome, età e stipendio, dove stipendio deve essere un attributo privato.

Costruire tutti i metodi getter e setter per gli attributi (anche per quelli pubblici)

Effettuare l'overriding del metodo __str__ in maniera tale che restituisca gli attributi nome e età.

Provare la classe istanziando almeno due oggetti.

Secondo Esercizio

Creare una classe Rettangolo con attributi base e altezza. Costruire tutti i metodi setter e getter per gli attributi. Aggiungere i metodi per il calcolo dell'area e del perimetro.

Implementare un metodo di nome: "contiene" che ha come parametro un oggetto rettangolo. Tale metodo deve restituire true se il rettangolo oggetto chiamante contiene il rettangolo oggetto parametro, false se non lo contiene. Un rettangolo "contiene" un altro quando la sua altezza e la sua base sono maggiori rispettivamente della base e dell'altezza del secondo rettangolo.

Terzo Esercizio

1 - Creare una classe Calcolo con un costruttore di default (senza parametri) che consenta di eseguire vari calcoli su numeri interi.

2 - Creare un metodo chiamato Factorial() che permetta di calcolare il fattoriale di un intero. Testare il metodo istanziando la classe.

3 - Creare un metodo chiamato Sum() che consenta di calcolare la somma dei primi n interi 1 + 2 + 3 + .. + n. Prova questo metodo.

4 - Creare un metodo tableMult() che crea e visualizza la tabellina di un dato intero. Quindi creare un metodo allTablesMult() per visualizzare tutte le tabelline di numeri interi 1, 2, 3, ..., 9.

Esercizi sull'ereditarietà

Quarto Esercizio

Si definisca una classe Persona che abbia i seguenti attributi:

- nome
- indirizzo
- età

Tale classe contiene i seguenti metodi: il costruttore, l'overriding del metodo __str__ e tutti i metodi getter e setter degli attributi.

Si vogliono derivare dalla classe Persona le seguenti classi:

- Studente
- Lavoratore

La prima deve avere gli attributi aggiuntivi:

- Scuola
- Media voti

La seconda deve avere gli attributi aggiuntivi:

- Azienda
- Stipendio

Aggiungere tutti i metodi getter e setter relativi agli attributi aggiuntivi.

Inoltre effettuare l'overriding dei costruttori e del metodo str inserendo gli attributi aggiuntivi.

Provare le tre classi instanziando almeno un oggetto per classe e provando qualche metodo.

Quinto Esercizio

Creare una classe AritmeticaDue con attributi operando1 e operando2. Definire il costruttore utilizzando parametri con valori predefiniti e il metodo str.

Aggiungere due metodi uno che restituisca la differenza e l'altro il prodotto dei due operandi. Implementare un terzo metodo che permetta il confronto tra il risultato del prodotto di due oggetti AritmeticaDue (in sostanza indicare se il prodotto è maggiore di quello calcolato nell'oggetto AritmeticaDue passato come parametro).

Derivare dalla classe AritmeticaDue la classe AritmeticaTre aggiungendo l'attributo operando3. Ridefinire il costruttore, il metodo str e i tre metodi differenza, prodotto e confronto. Aggiungere un metodo per il calcolo della somma di tutti gli attributi.

Provare le classi e i metodi implementati.

Esercizi sul polimorfismo e sull'import

Sesto Esercizio: Polimorfismo con oggetti veicolo

Si definisca la seguente:

1. Classe base Veicolo con:

   ➔ Costruttore che accetta marca e modello
   ➔ Metodo accendi() che restituisce la stringa:"Il veicolo si avvia"
   ➔ Metodo spegni() che restituisce una stringa:"Il veicolo si spegne"
   ➔ Metodo __str__() per la rappresentazione testuale

Successivamente:

2. Classe Auto che estende Veicolo con:

   ➔ Costruttore che accetta marca, modello e numero_porte
   ➔ Override del metodo accendi() che restituisce: "L'auto si avvia con la chiave"
   ➔ Override del metodo spegni() che restituisce: "L'auto si spegne girando la chiave"

3. Classe Moto che estende Veicolo con:

   ➔ Costruttore che accetta marca, modello e cilindrata
   ➔ Override del metodo accendi() che restituisce: "La moto si avvia premendo il pulsante"
   ➔ Override del metodo spegni() che restituisce: "La moto si spegne con l'interruttore"

4. Classe Bicicletta che estende Veicolo con:

   ➔ Costruttore che accetta marca, modello e tipo (es. "mountain bike", "city bike")
   ➔ Override del metodo accendi() che restituisce: "La bicicletta è sempre pronta!"
   ➔ Override del metodo spegni() che restituisce: "La bicicletta non si spegne mai"

5. Implementare la funzione gestisci_veicolo(veicolo) che:

   ➔ Stampa le informazioni del veicolo
   ➔ Chiama i metodi accendi() e spegni() sul veicolo ricevuto

6. Scrivere un programma principale che:

   ➔ Crea istanze di tutti i tipi di veicolo (parametri hard-coded)
   ➔ Le inserisce in una lista
   ➔ Itera sulla lista chiamando gestisci_veicolo() per ogni elemento

Esempio di output atteso:

```
Auto: Fiat Panda

L'auto si avvia con la chiave

L'auto si spegne girando la chiave

---

Moto: Honda CBR

La moto si avvia premendo il pulsante

La moto si spegne con l'interruttore

---

Bicicletta: Bianchi Mountain Bike

La bicicletta è sempre pronta!

La bicicletta non si spegne mai

---
```

Settimo Esercizio: Polimorfismo con operatori e collezioni eterogenee

Si vuole creare un programma che tratti i numeri complessi in forma algebrica e i vettori 2D. A tale scopo si definiscano le seguenti due classi:

1. Classe NumeroComplesso con:

- ➔ Costruttore che accetta parte_reale e parte_immaginaria
- ➔ Metodo __str__() che restituisce la forma forma algebrica: "a + bi" (se b <0 allora “a - |b|i)
- ➔ Metodo __add__() per sommare due numeri complessi. Regola: (a+bi) + (c+di) = (a+c) + (b+d)i
- ➔ Metodo __mul__() per moltiplicare due numeri complessi. Regola: (a+bi)(c+di) = (ac - bd) + (ad+bc)i
- ➔ Metodo modulo() che calcola il modulo del numero complesso. Regola: |a+bi| = √(a²+b²)

2. Classe Vettore2D con:

- ➔ Costruttore che accetta x e y
- ➔ Metodo __str__() che restituisce la forma "(x, y)"
- ➔ Metodo __add__() per sommare due vettori. Regola: (a,b)+(c,d) = (a+c , b+d)
- ➔ Metodo __mul__() per moltiplicare un vettore per uno scalare (numero). Regola: (a,b)*n = (an , bn)
- ➔ Metodo modulo() che calcola la lunghezza (o modulo) del vettore. | (a,b) | = √(a²+b²)

3. Implementare la funzione stampa_operazioni(obj1, obj2) che:

- ➔ Stampa i due oggetti
- ➔ Stampa la loro somma
- ➔ Stampa il modulo di entrambi

4. Scrivere un programma principale che:

- ➔ Crea istanze di NumeroComplesso e Vettore2D
- ➔ Testa le operazioni su ciascun tipo
- ➔ Crea una lista mista contenente numeri complessi e vettori
- ➔ Itera sulla lista calcolando e stampando il modulo di ogni elemento

Esempio di output atteso:

```
Numero complesso: 3 + 4i, modulo: 5.0

Numero complesso: 1 - 2i, modulo: 2.236

Somma: 4 + 2i

---

Vettore: (5, 12), modulo: 13.0

Vettore: (3, 4), modulo: 5.0

Somma: (8, 16)

---

Collezione mista:

Elemento: 3 + 4i, modulo: 5.0

Elemento: (5, 12), modulo: 13.0

Elemento: 1 - 2i, modulo: 2.236

---
```

Ottavo Esercizio: Sistema modulare per gestione biblioteca

Si vuole progettare un programma per la gestione di una biblioteca utilizzando l'importazione di classi da file separati.

Dopo aver modellizzato le classi che intervengono nel programma, si preferisce organizzare il codice in tre file separati:

File 1: media.py

Definisce le seguenti classi:

Classe base Media con:

- ➔ Costruttore che accetta titolo, autore, anno
- ➔ Metodo get_info() che restituisce le informazioni base
- ➔ Metodo __str__() per la rappresentazione

Classe Libro che eredita da Media con:

- ➔ Costruttore che accetta anche numero_pagine
- ➔ Override di get_info() che include il numero di pagine
- ➔ Metodo tempo_lettura() che calcola il tempo di lettura (1 pagina = 2 minuti)

Classe DVD che eredita da Media con:

- ➔ Costruttore che accetta anche durata_minuti
- ➔ Override di get_info() che include la durata
- ➔ Metodo tempo_visione() che restituisce la durata in formato "ore:minuti"

File 2: biblioteca.py

Definisce la classe:

Classe Biblioteca con:

- ➔ Costruttore che inizializza una lista vuota di media
- ➔ Metodo aggiungi_media(media) per aggiungere un elemento
- ➔ Metodo rimuovi_media(titolo) per rimuovere per titolo
- ➔ Metodo cerca_per_autore(autore) che restituisce una lista
- ➔ Metodo stampa_catalogo() che stampa tutti i media
- ➔ Metodo get_statistiche() che restituisce il numero di libri e DVD

- ➔ File 3: main.py

Programma principale che:

- ➔ Importa le classi necessarie dai moduli media e biblioteca
- ➔ Crea una biblioteca
- ➔ Aggiunge diversi libri e DVD
- ➔ Testa tutti i metodi implementati
- ➔ Stampa il catalogo completo e le statistiche

Struttura cartelle richiesta:

```
progetto/
|-- main.py
|-- media.py
|-- biblioteca.py
```

N.B. Non è necessario effettuare delle procedure di input da tastiera, le classi devono essere istanziate in modalità hard-coded.

Esempio di output atteso:

```
=== CATALOGO BIBLIOTECA ===
Libro: "1984" di George Orwell (1949) - 328 pagine
Tempo di lettura stimato: 656 minuti (10 ore e 56 minuti)
---
DVD: "Inception" di Christopher Nolan (2010) - 148 minuti
Tempo di visione: 2 ore e 28 minuti
---
Libro: "Il Nome della Rosa" di Umberto Eco (1980) - 512 pagine
Tempo di lettura stimato: 1024 minuti (17 ore e 4 minuti)

=== STATISTICHE ===
Libri: 2
DVD: 1
Totale media: 3

=== RICERCA PER AUTORE "Orwell" ===
Libro: "1984" di George Orwell (1949) - 328 pagine
```

Soluzione esercizi sulle classi (OOP)

Primo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Primo eEsercizio
    Creare una classe Insegnante con attributi nome, età
    e stipendio, dove stipendio deve essere un attributo
    privato. Costruire tutti i metodi getter e setter per
    gli attributi (anche quelli pubblici). Effettuare
    l'overriding del metodo __str__ in maniera tale che
    restituisca gli attributi nome e età.
    Provare la classe istanzando due oggetti.

"""


class Insegnante(object):
    """
    Classe Insegnante Es n.1
    La classe modellizza l'entità
    INSEGNANTE. Che ha tre attributi:
      nome: str
      età: int
      stipendio: float

    """

    def __init__(self, nome: str, eta: int, stipendio: float):
        """
        Costruttore dell'oggetto.
        Inizializza i 3 attributi grazie ai parametri
        formali in input al metodo stesso.
        L'attributo stipendio deve essere privato.
        """
        self.nome = nome
        self.eta = eta
        self.__stipendio = stipendio

    def getNome(self) -> str:
```

```
        """
        Getter dell'attributo nome
        """
        return self.nome

    def getEta(self) -> int:
        """
        Getter dell'attributo eta
        """
        return self.eta

    def getStipendio(self) -> float:
        """
        Getter dell'attributo stipendio
        """
        return self.__stipendio

    def setNome(self, val) -> None:
        """
        Setter dell'attributo nome
        """
        self.nome = val

    def setEta(self, val) -> None:
        """
        Setter dell'attributo eta
        """
        self.eta = val

    def setStipendio(self, val) -> None:
        """
        Setter dell'attributo stipendio
        """
        self.__stipendio = val

    def __str__(self):
        return (
            "Nome: "
            + str(self.nome)
            + " Eta': "
            + str(self.getEta())
            + " Stipendio: "
            + str(self.__stipendio)
        )
```

```
"""
Programma principale
Eseguiamo la prova della classe solo se il file.py
e' lanciato come programma. Se fosse importato
le istruzioni all'interno dell'if non vengono eseguite
"""

if __name__ == "__main__":
    ins1 = Insegnante("Paperino", 30, 2000)
    ins2 = Insegnante("Pippo", 43, 1500)

    print(ins1)
    print(ins2)
    ins1.setStipendio(3000)
    print(ins1.getStipendio())
```

Secondo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Secondo esercizio
    Creare una classe Rettangolo con attributi base e altezza.
    Costruire tutti i metodi setter e getter per gli attributi.
    Aggiungere i metodi per il calcolo dell'area e del perimetro.
    Implementare un metodo di nome: "contiene" che ha come
    parametro un oggetto  rettangolo. Tale metodo deve restituire
    true se il rettangolo oggetto chiamante contiene il rettangolo
    oggetto parametro, false se non lo contiene. Un rettangolo
    "contiene" un altro quando la sua altezza e la sua base sono
    maggiori rispettivamente della base e dell'altezza del secondo
    rettangolo.

"""


class Rettangolo(object):
    """
    Classe Rettangolo Es n.2
    La classe modellizza l'entità
    RETTANGOLO. Che ha due attributi:
        base: float
        altezza: float
    """

    def __init__(self, base: float = 1, altezza=1):
        """
        Costruttore dell'oggetto.
        Inizializza i 2 attributi grazie ai parametri
        formali in input al metodo stesso.
        Gli attributi sono privati.
        I parametri del metodo sono tutti predefiniti.
        """
        self.__base = base
        self.__altezza = altezza

    def getBase(self) -> float:
        """
        Getter dell'attributo base
```

```
        """
        return self.__base

    def getAltezza(self) -> float:
        """
        Getter dell'attributo altezza
        """
        return self.__altezza

    def setBase(self, val) -> None:
        """
        Setter dell'attributo base
        """
        self.__base = val

    def setAltezza(self, val) -> None:
        """
        Getter dell'attributo altezza
        """
        self.__altezza = val

    def getArea(self) -> float:
        """
        Metodo: getArea
        Metodo che restituisce l'area
        del rettangolo.
        Valore di ritorno:
        float -> area
        """
        return self.__base * self.__altezza

    def getPerimetro(self):
        """
        Metodo: getPerimetro
        Metodo che restituisce il perimetro
        del rettangolo.
        Valore di ritorno:
        float -> perimentro
        """
        return 2 * self.__base + 2 * self.__altezza

    def __str__(self):
        return "Base: " + str(self.__base) + " altezza: " +
str(self.__altezza)

    def contiene(self, rettangolo) -> bool:
```

```
        """
        Metodo: contiene
        Metodo che verifica se il rettangolo
        contiene un rettangolo passato come
        parametro al metodo.
        Valore di ritorno:
        bool -> True se contiene altrimenti False
        """
        ret = False
        if (
            rettangolo.getBase() < self.__base
            and rettangolo.getAltezza() < self.__altezza
        ):
            ret = True
        return ret


"""
Programma principale
Eseguiamo la prova della classe solo se il file.py
e' lanciato come programma. Se fosse importato
le istruzioni all'interno dell'if non vengono eseguite
"""

if __name__ == "__main__":
    rect1 = Rettangolo(2, 2)
    rect2 = Rettangolo()

    print(rect1)
    print(rect2)
    print(rect1.contiene(rect2))
    print(rect2.contiene(rect1))
```

Terzo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Terzo esercizio
    1 - Creare una classe Calcolo con un costruttore di default
    (senza parametri) che consenta di eseguire vari calcoli su
    numeri interi.
    2 - Creare un metodo chiamato Factorial() che permetta di
    calcolare il fattoriale di un intero. Testare il metodo
    istanziando la classe.
    3 - Creare un metodo chiamato Sum() che consenta di calcolare
    la somma dei primi n interi 1 + 2 + 3 + .. + n. Prova questo metodo.
    4 - Creare un metodo tableMult() che crea e visualizza la tabellina
    di un dato intero. Quindi creare un metodo allTablesMult() per
    visualizzare tutte le tabelline di numeri interi 1, 2, 3, ..., 9.

"""


class Calcolo(object):
    """
    Classe Calcolo Es n.3
    La classe modellizza l'entità
    CALCOLO. Non ha attributi.

    """

    def __init__(self):
        """
        Costruttore dell'oggetto.
        Non fa nulla di più del costruttore
        di default standard.
        """
        pass

    def factorial(self, numero: int) -> int:
        """
        Metodo: factorial
        Metodo che restituisce il
        fattoriale del numero
        passato come parametro.
```

```
        Parametri formali:
        int -> numero
        Valore di ritorno:
        int -> fattoriale
        bool -> True se errore altrimenti False
        """
        # Inizializzo errore
        err = True
        if numero >= 0:
            # Inizializzo fattoriale
            fatt = 1
            for x in range(1,numero + 1):
                fatt *= x
            err = False
        return fatt, err

    def sum(self, num: int) -> int:
        """
        Metodo: sum
        Metodo che restituisce la
        somma dei primi num numeri
        naturali.
        Parametri formali:
        int -> num
        Valore di ritorno:
        bool -> True se errore altrimenti False
        int -> somma
        """
        err = True
        somma = 0
        if num >= 0:
            for x in range(num + 1):
                somma += x
            err = False
        return somma, err

    def __tableMult(self, num: int) -> int:
        """
        Metodo: tableMult
        Metodo che restituisce la
        tabellina del numero num
        Parametri formali:
        int -> num
        Valore di ritorno:
        str -> tabellina di num
```

```
        """
        # Inizializzazione strnga di ritorno
        ret = "\n"
        for x in range(1, 11):
            ret += str(x * num) + " "
        return ret

    def allTableMult(self):
        """
        Metodo: allTableMult
        Metodo che restituisce la
        tabellina dei numeri da 1 a 9
        Parametri formali:
        None
        Valore di ritorno:
        str -> tabellina
        """
        ret = ""
        for x in range(1, 10):
            ret += self.__tableMult(x)
        return ret


"""
Programma principale
Eseguiamo la prova della classe solo se il file.py
e' lanciato come programma. Se fosse importato
le istruzioni all'interno dell'if non vengono eseguite
"""
if __name__ == "__main__":
    calc = Calcolo()

    print(calc.factorial(0))
    print(calc.factorial(1))
    print(calc.factorial(3))
    print(calc.sum(3))
    print(calc.allTableMult())
```

Soluzione esercizi sull'ereditarietà

Quarto Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Quarto Esercizio
    Si definisca una classe Persona che abbia i seguenti attributi:
        nome
        indirizzo
        età
    Tale classe contiene i seguenti metodi: il costruttore, l'overriding
    del metodo __str__ e tutti i metodi getter e setter degli attributi.
    Si vogliono derivare dalla classe Persona le seguenti classi:
        Studente
        Lavoratore
    La prima deve avere gli attributi aggiuntivi:
        Scuola
        Media voti
    La seconda deve avere gli attributi aggiuntivi:
        Azienda
        Stipendio
    Aggiungere tutti i metodi getter e setter relativi agli attributi
    aggiuntivi. Inoltre effettuare l'overriding dei costruttori e del
    metodo str inserendo gli attributi aggiuntivi.
    Provare le tre classi instanziando almeno un oggetto per classe e
    provando qualche metodo.

"""


class Persona(object):
    """
    Classe Persona Es n.4
    La classe modellizza l'entità
    Persona. Che ha tre attributi:
      nome: str
      età: int
      stipendio: float

    """
```

```
    def __init__(self, nome, indirizzo, eta):
        """
        Costruttore dell'oggetto.
        Inizializza i 3 attributi grazie ai parametri
        formali in input al metodo stesso.
        L'attributo stipendio deve essere privato.
        """
        self.__nome = nome
        self.__indirizzo = indirizzo
        self.__eta = eta

    def getNome(self):
        """
        Getter dell'attributo nome
        """
        return self.__nome

    def setNome(self, val):
        """
        Setter dell'attributo nome
        """
        self.__nome = val

    def getIndirizzo(self):
        """
        Getter dell'attributo indirizzo
        """
        return self.__indirizzo

    def setIndirizzo(self, val):
        """
        Setter dell'attributo indirizzo
        """
        self.__indirizzo = val

    def getEta(self):
        """
        Getter dell'attributo eta
        """
        return self.__eta

    def setEta(self, val):
        """
        Setter dell'attributo età
```

```
        """
        self.__eta = val

    def __str__(self):
        """
        Override del metodo __str__
        """
        return (
            "Nome: "
            + str(self.__nome)
            + " Eta': "
            + str(self.__eta)
            + " Indirizzo: "
            + str(self.__indirizzo)
        )


class Studente(Persona):
    """
    Classe Studente Es n.4
    La classe modellizza l'entità
    Studente estendendo la classa
    Persona. Aggiunge 2 nuovi attributi:
      scuola: str
      media: int
    """

    def __init__(self, nome: str, indirizzo: str, eta: int, scuola: str,
media: float):
        """
        Costruttore dell'oggetto.
        Inizializza i 5 attributi grazie ai parametri
        formali in input al metodo stesso.
        """
        super().__init__(nome, indirizzo, eta)
        self.__scuola = scuola
        self.__media = media

    def getScuola(self) -> str:
        """
        Getter dell'attributo scuola
        """
        return self.__scuola

    def setScuola(self, val: str) -> None:
        """
```

```
        Getter dell'attributo scuola
        """
        self.__scuola = val

    def getMedia(self) -> str:
        """
        Getter dell'attributo media
        """
        return self.__media

    def setMedia(self, val: float) -> None:
        """
        Getter dell'attributo media
        """
        self.__media = val

    def __str__(self) -> str:
        """
        Override del metodo __str__
        """
        return (
            super().__str__()
            + "\nScuola: "
            + str(self.__scuola)
            + " Media: "
            + str(self.__media)
        )


class Lavoratore(Persona):
    """
    Classe Lavoratore Es n.4
    La classe modellizza l'entità
    Studente estendendo la classa
    Persona. Aggiunge 2 nuovi attributi:
      azienda: str
      stipendio: float
    """

    def __init__(
        self, nome: str, indirizzo: str, eta: int, azienda: str,
stipendio: float
    ):
        """
        Costruttore dell'oggetto.
        Inizializza i 5 attributi grazie ai parametri
```

```
        formali in input al metodo stesso.
        """
        super().__init__(nome, indirizzo, eta)
        self.__azienda = azienda
        self.__stipendio = stipendio

    def getAzienda(self) -> str:
        """
        Getter dell'attributo azienda
        """
        return self.__azienda

    def setAzienda(self, val: str) -> None:
        """
        Getter dell'attributo azienda
        """
        self.__azienda = val

    def getStipendio(self) -> float:
        """
        Getter dell'attributo stipendio
        """
        return self.__stipendio

    def setStipendio(self, val: float) -> None:
        """
        Getter dell'attributo stipendio
        """
        self.__stipendio = val

    def __str__(self) -> str:
        """
        Override del metodo __str__
        """
        return (
            super().__str__()
            + "\nAzienda: "
            + str(self.__azienda)
            + " Stipendio: "
            + str(self.__stipendio)
        )


"""
Programma principale
```

```
Eseguiamo la prova delle classi solo se il file.py
e' lanciato come programma. Se fosse importato
le istruzioni all'interno dell'if non vengono eseguite
"""

if __name__ == "__main__":
    p1 = Persona("Pippo", "Via Tale dei Tali", 20)
    print(p1)
    print()
    s1 = Studente("Paolo", "Via XXX", 15, "Liceo Roma", 8)
    print(s1)
    print()
    l1 = Lavoratore("Pietro", "Corso Roma", 45, "Stellantis", 2000)
    print(l1)
    print()
```

Quinto Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Quinto Esercizio
    Creare una classe AritmeticaDue con attributi operando1
    e operando2. Definire il costruttore utilizzando parametri
    con valori predefiniti e il metodo str.
    Aggiungere due metodi uno che restituisca la differenza
    e l'altro il prodotto dei due operandi. Implementare un
    terzo metodo che permetta il confronto tra il risultato
    del prodotto di due oggetti AritmeticaDue (in sostanza
    indicare se il prodotto è maggiore di quello calcolato
    nell'oggetto AritmeticaDue passato come parametro).
    Derivare dalla classe AritmeticaDue la classe AritmeticaTre
    aggiungendo l'attributo operando3. Ridefinire il costruttore,
    il metodo str e i tre metodi differenza, prodotto e confronto.
    Aggiungere un metodo per il calcolo della somma di tutti gli
    attributi.
    Provare le classi e i metodi implementati.

"""


class AritmeticaDue(object):
    """
    Classe AritmeticaDue Es n.5
    La classe modellizza l'entità
    ARITMETICADUE. Che ha due attributi:
      operando1: float
      operando2: float

    """

    def __init__(self, operando1: float = 0, operando2: float = 0):
        """
        Costruttore dell'oggetto.
        Inizializza i 2 attributi grazie ai parametri
        formali in input al metodo stesso.
        """
        self._operando1 = operando1
        self._operando2 = operando2
```

```
def __str__(self) -> str:
    """
    Override del metodo __str__
    """
    return (
        "operando1: "
        + str(self._operando1)
        + " operando2': "
        + str(self._operando2)
    )

def prodotto(self) -> float:
    """
    Metodo prodotto.
    Esegue il prodotto degli attributi operando1
    e operando2.
    Valore di ritorno:
    float -> prodotto degli attributi
    """
    return self._operando1 * self._operando2

def differenza(self) -> float:
    """
    Metodo differenza.
    Esegue la differenza degli attributi operando1
    e operando2.
    Valore di ritorno:
    float -> operando1- operando2
    """
    return self._operando1 - self._operando2

def confronta(self, oggettoArDue) -> bool:
    """
    Metodo confronta.
    Esegue il confronto tra se stesso e
    l'oggetto aritmeticaDue passato come
    parametro formale. Se il prodotto è maggiore
    allora si restutuisce True altrimenti False.
    Parametri formali:
    AritmeticaDue -> oggetto su cui effettuare il confronto
    Valore di ritorno:
    bool -> True se prodotto maggiore altrimenti False
    """
    return self.prodotto() > oggettoArDue.prodotto()
```

```
class AritmeticaTre(AritmeticaDue):
    """
    Classe AritmeticaTre Es n.5
    La classe modellizza l'entità
    ARITMETICATRE derivadola dalla
    classe ARITMETICADUE. Aggiungendo
    un attributo:
      operando3: float

    """

    def __init__(
        self, operando1: float = 0, operando2: float = 0, operando3:
float = 0
    ):
        """
        Costruttore dell'oggetto.
        Inizializza i 3 attributi grazie ai parametri
        formali in input al metodo stesso.
        """
        super().__init__(operando1, operando2)
        self._operando3 = operando3

    def __str__(self) -> str:
        """
        Override del metodo __str__
        """
        return super().__str__() + " operando3: " + str(self._operando3)

    def prodotto(self) -> float:
        """
        Metodo prodotto.
        Override  di super().prodotto
        Esegue il prodotto degli attributi operando1,
        operando2 e operando3.
        Valore di ritorno:
        float -> prodotto degli attributi
        """
        return super().prodotto() * self._operando3

    def differenza(self) -> float:
        """
        Metodo differenza.
        Override  di super().differenza
```

```
        Esegue il seguente calcolo aritmetico:
        operando1 - operando2 -operando3
        Valore di ritorno:
        float -> risultato operazione
        """
        return self._operando1 - self._operando2 - self._operando3

    def somma(self) -> float:
        """
        Metodo somma.
        Esegue la somma di tutti e tre gli
        attrinuti.
        Valore di ritorno:
        float -> risultato somma
        """
        return sum([self._operando1, self._operando2, self._operando3])


"""
Programma principale
Eseguiamo la prova delle classi solo se il file.py
e' lanciato come programma. Se fosse importato
le istruzioni all'interno dell'if non vengono eseguite
"""

if __name__ == "__main__":
   d1=AritmeticaDue(1,2)
   d2=AritmeticaDue(2,3)
   print(d1,d2)
   print(d1.confronta(d2))
   t1=AritmeticaTre(1,2,3)
   t2=AritmeticaTre(1,2,4)
   print(t1,t2)
   print(t1.confronta(t2))
   print(t2.confronta(t1))
```

Soluzione esercizi sul polimorfismo e sull'import

Sesto Esercizio

```
# ------------------------------------------------------
# Esercizi OOP
# es. 8.6
#
# ------------------------------------------------------

class Veicolo:
    """Classe base per tutti i veicoli"""

    def __init__(self, marca:str, modello:str):
        self.marca = marca
        self.modello = modello

    def accendi(self) -> str :
        return "Il veicolo si accende"

    def spegni(self) -> str :
        return "Il veicolo si spegne"

    def __str__(self):
        return f"{self.__class__.__name__}: {self.marca} {self.modello}"


class Auto(Veicolo):
    """Classe per le automobili"""

    def __init__(self, marca:str, modello:str, numero_porte:int):
        super().__init__(marca, modello)
        self.numero_porte = numero_porte

    def accendi(self) -> str :
        return "L'auto si avvia con la chiave"

    def spegni(self) -> str :
        return "L'auto si spegne girando la chiave"


class Moto(Veicolo):
```

```
    """Classe per le motociclette"""

    def __init__(self, marca, modello, cilindrata):
        super().__init__(marca, modello)
        self.cilindrata = cilindrata

    def accendi(self) -> str :
        return "La moto si avvia premendo il pulsante"

    def spegni(self) -> str :
        return "La moto si spegne con l'interruttore"


class Bicicletta(Veicolo):
    """Classe per le biciclette"""

    def __init__(self, marca, modello, tipo):
        super().__init__(marca, modello)
        self.tipo = tipo

    def accendi(self) -> str :
        return "La bicicletta è sempre pronta!"

    def spegni(self) -> str :
        return "La bicicletta non si spegne mai"

    def __str__(self) -> str :
        return f"{self.__class__.__name__}: {self.marca} {self.tipo}"


def gestisci_veicolo(veicolo) -> None :
    """Funzione che dimostra il polimorfismo"""
    print(veicolo)
    print(veicolo.accendi())
    print(veicolo.spegni())
    print("---")


# Programma principale Esercizio 1
if __name__ == "__main__":
    # Creazione istanze
    auto = Auto("Fiat", "Panda", 5)
    moto = Moto("Honda", "CBR", 600)
    bici = Bicicletta("Bianchi", "Mountain Bike", "mountain bike")
```

```
# Lista di veicoli diversi
veicoli = [auto, moto, bici]

# Polimorfismo in azione
print("=== GESTIONE VEICOLI ===")
for veicolo in veicoli:
    gestisci_veicolo(veicolo)
```

Settimo Esercizio

```
'''
Esercizi OOP
es. 8.7

'''

import math


class NumeroComplesso:
    """Classe per rappresentare numeri complessi"""

    def __init__(self, parte_reale: float, parte_immaginaria: float):
        self.reale = parte_reale
        self.immaginaria = parte_immaginaria

    def __str__(self) -> str:
        if self.immaginaria >= 0:
            return f"{self.reale} + {self.immaginaria}i"
        else:
            return f"{self.reale} - {abs(self.immaginaria)}i"

    def __add__(self, other: object) -> object:
        """Somma di due numeri complessi"""
        return NumeroComplesso(
                        self.reale  +  other.reale,  self.immaginaria  +
other.immaginaria
        )

    def __mul__(self, other: object) -> object:
            """Moltiplicazione  di  due  numeri  complessi:  (a+bi)(c+di)  =
(ac-bd) + (ad+bc)i"""
            reale  =  self.reale  *  other.reale  -  self.immaginaria  *
other.immaginaria
            immaginaria = self.reale * other.immaginaria + self.immaginaria
* other.reale
        return NumeroComplesso(reale, immaginaria)

    def modulo(self) -> float:
        """Calcola il modulo del numero complesso"""
        return math.sqrt(self.reale**2 + self.immaginaria**2)


class Vettore2D:
    """Classe per rappresentare vettori 2D"""
```

```
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __str__(self) -> str:
        return f"({self.x}, {self.y})"

    def __add__(self, other: object) -> object:
        """Somma di due vettori"""
        return Vettore2D(self.x + other.x, self.y + other.y)

    def __mul__(self, scalare: float) -> object:
        """Moltiplicazione del vettore per uno scalare"""
        return Vettore2D(self.x * scalare, self.y * scalare)

    def modulo(self) -> float:
        """Calcola la lunghezza del vettore"""
        return math.sqrt(self.x**2 + self.y**2)


def stampa_operazioni(obj1:Vettore2D, obj2:Vettore2D)-> None:
    """Funzione che dimostra il polimorfismo con oggetti diversi"""
    print(f"Primo oggetto: {obj1}")
    print(f"Secondo oggetto: {obj2}")
    print(f"Somma: {obj1 + obj2}")
    print(f"Modulo primo: {obj1.modulo():.3f}")
    print(f"Modulo secondo: {obj2.modulo():.3f}")
    print()


# Programma principale
if __name__ == "__main__":
    print("=== NUMERI COMPLESSI ===")
    c1 = NumeroComplesso(3, 4)
    c2 = NumeroComplesso(1, -2)
    stampa_operazioni(c1, c2)

    print("=== VETTORI 2D ===")
    v1 = Vettore2D(5, 12)
    v2 = Vettore2D(3, 4)
    stampa_operazioni(v1, v2)

    print("=== COLLEZIONE MISTA ===")
    oggetti_misti = [
        NumeroComplesso(3, 4),
```

```
    Vettore2D(5, 12),
    NumeroComplesso(1, -2),
    Vettore2D(8, 6),
]

# Loop sulla lista
for obj in oggetti_misti:
    print(f"Elemento: {obj}, modulo: {obj.modulo():.3f}")
```

Ottavo Esercizio

```
"""
Esercizi OOP
es. 8.8
```

FILE: media.py

```
"""

class Media:
    """Classe base per tutti i tipi di media"""

    def __init__(self, titolo, autore, anno):
        self.titolo = titolo
        self.autore = autore
        self.anno = anno

    def get_info(self):
        return f'"{self.titolo}" di {self.autore} ({self.anno})'

    def __str__(self):
        return self.get_info()


class Libro(Media):
    """Classe per i libri"""

    def __init__(self, titolo, autore, anno, numero_pagine):
        super().__init__(titolo, autore, anno)
        self.numero_pagine = numero_pagine

    def get_info(self):
        return f'Libro: "{self.titolo}" di {self.autore} ({self.anno}) - {self.numero_pagine} pagine'

    def tempo_lettura(self):
        """Calcola tempo di lettura in minuti (1 pagina = 2 minuti)"""
        minuti = self.numero_pagine * 2
        ore = minuti // 60
        minuti_restanti = minuti % 60
        return f"{minuti} minuti ({ore} ore e {minuti_restanti} minuti)"


class DVD(Media):
```

```
    """Classe per i DVD"""

    def __init__(self, titolo, autore, anno, durata_minuti):
        super().__init__(titolo, autore, anno)
        self.durata_minuti = durata_minuti

    def get_info(self):
        return f'DVD: "{self.titolo}" di {self.autore} ({self.anno}) - {self.durata_minuti} minuti'

    def tempo_visione(self):
        """Restituisce durata in formato ore:minuti"""
        ore = self.durata_minuti // 60
        minuti = self.durata_minuti % 60
        return f"{ore} ore e {minuti} minuti"


"""
Esercizi OOP
es. 8.8
```

FILE: biblioteca.py

```
"""

from media import Media, Libro, DVD


class Biblioteca:
    """Classe per gestire una biblioteca di media"""

    def __init__(self):
        self.catalogo = []

    def aggiungi_media(self, media: Media) -> None:
        """Aggiunge un media al catalogo"""
        self.catalogo.append(media)
        print(f"Aggiunto: {media.get_info()}")

    def rimuovi_media(self, titolo: str) -> bool:
        """Rimuove un media dal catalogo per titolo"""
        ret = False
        for media in self.catalogo:
            if media.titolo.lower() == titolo.lower():
                self.catalogo.remove(media)
                print(f"Rimosso: {media.get_info()}")
```

```
                ret = True
                break
        else:
            print(f"Media con titolo '{titolo}' non trovato")
        return ret

    def cerca_per_autore(self, autore) -> list:
        """Cerca tutti i media di un autore"""
        risultati = []
        for media in self.catalogo:
            if autore.lower() in media.autore.lower():
                risultati.append(media)
        return risultati

    def stampa_catalogo(self) -> None:
        """Stampa tutto il catalogo"""
        print("=== CATALOGO BIBLIOTECA ===")

        if self.catalogo:  # verifico che il catalo non sia vuoto
            for media in self.catalogo:
                print(media.get_info())
                # Stampa informazioni aggiuntive specifiche per tipo
                if hasattr(media, "tempo_lettura"):  # È un libro
                                print(f"Tempo di lettura stimato:
{media.tempo_lettura()}")
                elif hasattr(media, "tempo_visione"):  # È un DVD
                    print(f"Tempo di visione: {media.tempo_visione()}")
                print()
        else:
            print("Il catalogo è vuoto")

    def get_statistiche(self) -> dict:
        """Restituisce statistiche del catalogo"""
          libri = sum(1 for media in self.catalogo if isinstance(media,
Libro))
           dvd = sum(1 for media in self.catalogo if isinstance(media,
DVD))
                    return {"libri": libri, "dvd": dvd, "totale":
len(self.catalogo)}

"""
Esercizi OOP
es. 8.8
```

FILE: main.py

```
"""
```

```
from media import Libro, DVD
from biblioteca import Biblioteca


# Programma principale
print("=== SISTEMA BIBLIOTECA ===\n")

# Creazione biblioteca
biblioteca = Biblioteca()

# Aggiunta di alcuni media
print("--- Aggiunta Media ---")
libro1 = Libro("1984", "George Orwell", 1949, 328)
libro2 = Libro("Il Nome della Rosa", "Umberto Eco", 1980, 512)
dvd1 = DVD("Inception", "Christopher Nolan", 2010, 148)
dvd2 = DVD("Interstellar", "Christopher Nolan", 2014, 169)

biblioteca.aggiungi_media(libro1)
biblioteca.aggiungi_media(libro2)
biblioteca.aggiungi_media(dvd1)
biblioteca.aggiungi_media(dvd2)
print()

# Stampa catalogo completo
biblioteca.stampa_catalogo()

# Statistiche
stats = biblioteca.get_statistiche()
print("=== STATISTICHE ===")
print(f"Libri: {stats['libri']}")
print(f"DVD: {stats['dvd']}")
print(f"Totale media: {stats['totale']}")
print()

# Ricerca per autore
print('=== RICERCA PER AUTORE "Nolan" ===')
risultati = biblioteca.cerca_per_autore("Nolan")
for media in risultati:
    print(media.get_info())
print()

# Test rimozione
print("--- Test Rimozione ---")
biblioteca.rimuovi_media("Interstellar")
```

```
# Stampa statistiche finali
stats_finali = biblioteca.get_statistiche()
print(f"\nStatistiche finali - Totale media: {stats_finali['totale']}")
```